46-926, Spring 2019: Homework #3

Due **5PM**, February 11, 2019.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

Some comments on the problems: Problem 1, 3, 5 are reasonably brief. Problems 2 and 4 are longer. There is a last, ungraded problem at the end that you can do for fun – the promised math behind smoothing splines – but which we will not collect or grade. I had too many questions that I thought would be useful, so I had to drop this one. I still encourage you to work it out sometime when you have time, as it is a useful idea to understand.

1. *You actually saw this data in Data Science as an example of kernel regression, but we will use it again to look at smoothing splines.*

   One approach to estimating the forward rate function (which can be used to calculate bond prices and yield curves) is to obtain many noisy point estimates of the forward rate at different times by comparing pairs of zero-coupon bonds, and them smooth them to get a more stable estimate of the curve.

   (a) Load `forward_rates.csv`. This contains a set of empirical forward rate estimates for a range of times (courtesy of Ruppert and Matteson, 2015, which you can also read – free on SpringerLink – for a better description of the data). Plot the empirical rates against time.
   You should see that the points make a very noisy estimate of a smooth curve. It should be clear that a linear function, or even a quadratic one, would not capture this shape very well.

   (b) Fit a smoothing spline to this data. In python, use LinearGAM like we did in class. In R, use `smooth.spline`. (Up to you.) Cross-validate for a reasonable lambda. Plot your fit on the same plot as the points. Can you manage to get a believable, smooth fit?

(c) Computing the yield curve from this forward rate function involves definite integrals of your estimated smooth function. Splines are particularly nice for doing this. Think about the nature of cubic splines, and explain briefly why they can be integrated quickly and in essentially closed form (no numeric integration).

To think about: Compare this to nonparametric estimates from kernel regression, where you would need to evaluate the function on a fine grid of points and then carry out numerical integration.

2. This problem explores additive models for prediction. You will consider a bike rental data set, and forecast utilization based on time and weather. This information could be useful for tasks like inventory management and maintenance; we will explore this impact in terms of loss later in the problem.

   Download `bike_train.csv` and `bike_test.csv` from canvas. These have daily data for two years. Load the data and get familiar with it.

   - `dayofyear`, `dayofweek` are the 0-indexed day of the year and of the week. I've pretended that 2012 is not a leap year.
   - `workday` is a binary indicator of whether it is a workday (1) or a weekend/holiday (0).
   - `weathertype` is a four level variable for type of weather, with higher numbers indicating worse weather: 1: Clear, Few clouds, Partly cloudy, Partly cloudy; 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist; 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds; 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
   - `temp, humidity, windspeed` are all what they sound like.
   - `rentals` is the number of rentals, your target variable for prediction.

   Important: We will be fitting on data from 2011 and testing on data from 2012. However, this service became much more popular in 2012. To give your methods a fighting chance, you get the additional information that your overall ridership is up about 1.6 times. That is, *whenever you make a prediction on the test data, multiply your prediction by 1.6.*

   Note: You will be fitting a few different models. Store each of them as you go, since we will evaluate them in different ways later in the problem.

   (a) Suppose that you want to predict the number of rentals (`rentals`) based on `dayofyear, dayofweek, workday, weathertype, temp, humidity, windspeed`. Fit a *linear model* to make this prediction.

Make predictions on your test data. Remember to scale your guesses by 1.6 to account for overall growth. What is your MSE on the test data?

(b) Now fit an additive model. Note that smooth fits make sense for some of your variables, but not others. However, pygam's defaults handle this fine so you don't need to worry.

If you're using MGCV in R, you should wrap the following terms in `s()` in your formula to make them smooth: `dayofyear, temp, humidity, windspeed`.

Make predictions on your test data (again scaling by 1.6). What is your MSE now?

(c) Plot the functions for each of your smooth covariates. What nonlinear trends do you see that a linear model would miss? Give a very brief interpretation for each of your smooth fits.

(d) It turns out that MSE is a strange measure for our inventory management. Consider the following more realistic loss function (though also idealized): For every unit that we are understocked, we lose \$5 because we miss out on a rental. For every unit that we overstock, we lose \$1 because we maintain a surplus inventory:

$$\mathcal{L}(Y, \widehat{Y}) = \begin{cases} 5(Y - \widehat{Y}) & \text{if } Y > \widehat{Y} \\ \widehat{Y} - Y & \text{if } Y < \widehat{Y} \end{cases}$$

The following function computes this new loss:

```
def real_loss(true, guess):
    return(np.mean(np.where(true>guess, 5*(true-guess), guess-true)))
```

What is this loss for your linear and your spline model?

(e) As you saw on your last homework, when the loss is asymmetric, the mean is not a great predictor. One solution would be to optimize the correct loss (in this case, the corresponding method is called *quantile regression*). Instead, let's consider adjusting our estimates with a hack.

Since it hurts more to undershoot than overshoot, we want to skew our estimates upward to improve our loss. It turns out that a reasonable adjustment would be to add the 83.333% (5/6) quantile of our residual distribution to our estimates. (We will discuss this further in class.) Because of the weird scaling across years and to simplify this homework, we'll use the test data to calculate this quantile. Other approaches are possible.

Using the `test` data, calculate the 5/6th quantile of the residuals $Y - \widehat{Y}$ for each predictor (where you have scaled $\widehat{Y}$ by 1.6 already). In python, you can use `np.percentile`; in `R`, you can use `quantile`. Now build new predictors for both your linear method and your additive model by adding the corresponding quantile to your predictions. That is:

$$\widehat{Y}_{new} = \widehat{Y} + \text{quantile}$$

Evaluate these new predictions on the test set in terms of the asymmetric loss function. How does the performance compare to the unadjusted estimator from the previous part? How does the additive model compare to the linear model now?

What would be your final choice of estimator?

3. *Best classification when your losses are asymmetric.* Consider a two-class ($\{0, 1\}$) classification problem. We saw in class that for 0-1 loss, the optimal classification rule is

$$f(x) = \begin{cases} 1 & \text{if } P\left(Y = 1 \mid X = x\right) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

if we assume that we know everything about the distribution of (X, Y).

Suppose instead that our loss is

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	L_{01}
Guess $Y = 0$	L_{10}	0

(a) Again assuming that we know everything about the distribution, find the best possible classification function $f(x)$ for minimizing this new, more general loss. The calculation will be very similar to the one in class.

(b) If $L_{10} > L_{01}$ (so that false negatives are worse than false positives), how does this classification rule differ from the 0-1 loss rule we derived in class? Why does this make sense? (You can just answer with two short sentences.)

(c) Suppose that you are constructing your classifier using logistic regression. Before, we had a classification rule of

$$\widehat{f}(x) = \begin{cases} 1 & \text{if } x^T\widehat{\beta} > 0 \\ 0 & \text{otherwise} \end{cases}$$

What should this rule become under the new loss? What does this do to the linear decision boundary when $L_{10} > L_{01}$?

To think about: How should your classifiers change if your class frequencies are different between your training and test data?

4. *Prediction with marketing data* For this problem, we will be working with marketing data from a bank, found in `marketing.csv` on blackboard. We observe the results from a calling campaign that attempted to sell a new product to clients. Based on the covariates of the client, we want to build a classifier to predict whether the client will invest in the product.

   We observe the following variables about the client:

   - `age`
   - `job`: Job classification
   - `marital`: Marital status
   - `education`: Level of education
   - `default`: Currently in default?
   - `balance`: Average yearly account balance
   - `housing`: Currently has a housing loan?
   - `loan`: Currently has a personal loan?

   You also observe a variable `y` which is either `no` or `yes`, indicating whether the sale was successful. This is your outcome variable.

   You can load the data into `pythohn` with the `read_csv` command. Familiarize yourself with the variables and data types in this data set.

   Hint: You will notice that we have several categorical variables. In sklearn's logistic regression, these are not handled automatically like they are in `R`. Instead, you need to convert them to dummy variables (also called one-hot encoding. You can do this by calling the `pd.get_dummies` function on your data frame, which returns a dummy-encoded version of the data frame.

   Similarly, you can recode your `y` variable as $\{0, 1\}$ with

```
y = np.where(dat['y'] == 'yes',1,0)
```

or just use a column returned from the dummy call.

We will also split the data into a training set and a test set. For uniformity and ease of grading, use the following code to generate your training and test data:

```
train_test_split(X, y, test_size=0.33, random_state=1)
```

This splits off 33% of the data to be used as a validation set.

(a) Using the training data, estimate the fraction of successes (your *base rate*). This will be useful to keep in mind.

(b) Using your training data, fit a logistic regression using all of the variables to predict the outcome. Use `LogisticRegression` from `sklearn`, with `C=100000` to downweight the ridge penalty. What is your misclassification rate (fraction of wrong guesses in any direction) on the test set, using the usual logistic regression classification rule?

Hint: You can get three different kinds of "predictions" from logistic regression. Suppose that `fit` is the object returned by a call to `LogisticRegresssion().fit()`. Then:

- `fit.predict` gives class labels in $\{0, 1\}$
- `fit.predict_proba` gives probabilities on the p(x) scale. Note that this returns probabilities of both class 0 and class 1, so

  `fit.predict_proba(X_test)[:,1]`

  corresponds to the vector of $P(Y = 1|X = x_i)$.
- `fit.predict_log_proba` similarly gives the log probabilities. This is sometimes more convenient.

(c) If you used the silly classifier that guesses `'no'` for all observations, what would your misclassification rate be on the test data? How does this compare to the misclassification rate of your logistic classifier?

This might be rather discouraging for our classifier, if our true goal is minimizing 0-1 loss.

(d) Suppose that, despite this hopelessness, you still feel the need to call some clients. Using your logistic model, find the 1000 clients in the test set that are most likely to score `'yes'`. (That is, the 1000 observations from the test set with the highest predicted probability of $Y = 1$.) For this set of 1000 clients, what fraction of them actually say yes? What fraction would you expect if you picked a random set of 1000 clients?

Suddenly our model feels a little more useful!

> **Hint:** You may find the `np.argsort` and/or `np.flip` functions useful.

(e) Even though our simple classifier doesn't win in terms of 0-1 loss, we see that it helps to sort our points according to their likelihood of being `'yes'`. This can be a much more reasonable goal than misclassification error in heavily imbalanced samples. You can think of this as an approach to selecting an "enriched" set.

As you change the size of your selected set (500, 1000, 2000, etc), you change both:

- The fraction of true positives in the data that you manage to capture (called *sensitivity*)
- The fraction of true negatives you manage to avoid (called *specificity*)

When the set gets larger, sensitivity will necessarily increase, but specificity will decrease. The better your classifier, the more efficiently it makes this trade off.

An easy way to assess the quality of your classifier in this sense is to plot sensitivity versus specificity. A better classifier will tend to have a higher curve (better sensitivity for a given specificity). These curves (called ROC curves) have other nice properties that we will discuss in class; This problem is just intended as an introduction.

The `roc_curve` function from `sklearn.metrics` will compute TPR (the same as sensitivity) and FPR (the same as 1-specificity). It returns a tuple, of which the first two elements are FPR and TPR. Example usage:

```
fpr, tpr, _ = roc_curve(true_y_values, predicted_probabilities)
```

(The _ discards the third returned value, which we don't care about yet)

Using the `roc_curve` function, plot the TPR (y axis) against the FPR (x axis). This is your ROC curve.

If you classified by guessing randomly, on average you would just get the diagonal $y = x$ line; if your classifier is better than random guessing it should give a curve above this line. Include a diagonal $y = x$ line in your plot for comparison.

We will talk more about these measures and plots in our next class.

5. *Marketing data, continued.* Now we will try to improve your marketing classifier. Start where you left off in the last problem, using the same data sets.

   (a) Two of our variables, balance and age, are continuous. Furthermore, we doubt that the log odds of successfully advertising do not actually grow linearly in either of these variables. Fit a logistic GAM to the data, using smoothing splines to model the effect of balance and age. Plot the partial dependence plots for these two variables.

   (b) Evaluate the predictions from your logistic GAM in terms of misclassification. Also calculate the fraction of successes in the top ranked 1000 test observations. How does your performance on each of these metrics compare to logistic regression?

   (c) Make a new ROC curve plot. This plot should have three curves: (i) the straight $y = x$ line; (ii) The ROC curve for logistic regression from the previous problem; (iii) The ROC curve for your new logistic GAM.

[UNGRADED, JUST FOR FUN] *Understanding how splines are fit.* This problem is intended to walk you through a subset of the claims I made in class. Consider a smoothing spline fit to n points, $(x_1, y_1), \ldots, (x_n, y_n)$. For a fixed λ, we solve

$$\min_f \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int \left(f''(x)\right)^2 dx$$

Suppose that you know:

- The solution to such a problem must be a natural cubic spline with knots at the x_i.
- Natural cubic splines can be represented in terms of a basis of known functions of x:

  $$f(x) = \beta_0 + \sum_{k=1}^{K} \beta_k b_k(x)$$

  Here the $b_k(x)$ are the known basis functions.

This means that to fit your f, you only need to figure out the β_k.

1. Since we know each of the $b_k(x)$ functions, we could evaluate them at all of our training coordinates x_i. Show that you can rewrite the first part of your optimization objective, $\sum_{i=1}^{n}(y_i - f(x_i))^2$ entirely in terms of the y_i, $b_k(x_i)$, and β_k. Write the term entirely in matrix/vector notation, as we do in regression. You will need to define a matrix of your $b_k(x_i)$ values.

2. Similarly, rewrite your penalty term, $\int \left(f''(x)\right)^2 dx$ in terms of your functions $b_k(x)$ (and/or their derivatives), β_k. Convert this to a matrix/vector expression.

   Hint: You can assume you know all the derivatives of your $b_k(x)$. Your matrices are allowed to have elements that depend on integrals of your $b_k(x)$ functions (and their derivatives). It may help to first think about what $f''(x)$ looks like in terms of the $b_k(x)$ functions, and then move along to squaring and integrating it.

3. Combining these two pieces, you should now have an objective that is written entirely in terms of matrices and vectors. The only unknowns will be your vector of β coefficients. If everything has gone right, your objective will now remind you very strongly of ridge regression.

   Thinking back to your homework from last week, find a solution closed form solution for β (in terms of the matrices you defined above).

The result from this problem means that you can calculate your coefficient β as a linear operator on your vector of Y. Once you have this β, your spline would just be $\beta_0 + \sum_{k=1}^{K} \beta_k b_k(x)$.